

Open in app ↗

Sign up

Sign in

Medium

 Search

Debugging TkInter application using X11 Forwarding and Remote Interpreter

4 min read · Aug 14, 2019



Reza Teshnizi

Follow



Listen



Share

Once upon a time, I was working on a python application and used TkInter to create a GUI demo. Everything was good until I had to use a python module that was only available on Linux. The problem was that in my lab, I have a Linux (Ubuntu) machine but my main laptop is a Surface Book running Windows 10. You see where this is going: I could no longer commit my code on my Ubuntu machine and continue working on it on my laptop.

In this article I will explain how you can get setup to run your code remotely, and optionally debug it using PyCharm.

Prerequisites

On Windows

1. **Git for Windows:** It comes with Git Bash and ssh
2. **PyCharm (Optional):** the reason I say it's optional is that PyCharm supports remote interpreter only with a Professional or Student license.

On Ubuntu

1. **sudo access**
2. **Python3:** you don't need python3! But I haven't tested this with python2 and who needs an angry mob!

Setting up Your Ubuntu Machine

1) Installing X11 Server

Chances are you installed your Ubuntu with GUI, but if you haven't you need to install `xauth` and `xorg`

```
sudo apt-get install xauth xorg
```

2) Setting Up SSH Server

Install ssh client (most likely already installed) and server

```
sudo apt-get update  
sudo apt-get install openssh-server
```

and make sure the ssh service is running

```
sudo systemctl status ssh
```

you should see something like

```
● ssh.service - OpenBSD Secure Shell server  
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)  
   Active: active (running) since Fri 2019-08-02 11:11:02 CDT; 4 days ago  
 Main PID: 1069 (sshd)  
    Tasks: 1 (limit: 4915)  
   CGroup: /system.slice/ssh.service  
           └─1069 /usr/sbin/sshd -D
```

Output of ssh status check

Now you should be able login to your Ubuntu machine using its IP from other machines.

```
ssh <username>@<ip_address>
```

Note that if your router or firewall has restriction on the ssh port, you would have to troubleshoot that. Moreover, you need to be on the same LAN or use a VPN to be able to access the IP.

3) Enable X11 Forwarding

Open `/etc/ssh/sshd_config` in your favorite editor and make sure this line is not commented:

```
X11Forwarding yes
```

With this you are now ready to move on to your Windows machine.

Setting up Your Windows Machine

1) Install Xming

To be able to display your GUI that is running remotely on Linux, you need an X Server on your Windows Machine. The go-to solution on Windows is Xming. Download the latest version and install it and run Xming once it's installed.

2) Test Your X11 setup

Now it's time to test this setup. Open Git Bash. Now before we run any GUI application, we need to tell the Xming Server where to display the GUI ([read more about the DISPLAY environment variable](#)):

```
export DISPLAY=localhost:0.0
```

Most likely, `localhost:0.0` is the right value. But in rare cases `0.0` are not the right display and screen number. But just to make sure you can hover over the little Xming tray icon to see the numbers.



Display and Screen number

Finally, you need to open connect to your Ubuntu machine using ssh. You should now be able to connect to your Ubuntu machine:

```
ssh -Y <username>@<ip_address>
```

-Y flag is important to allow X11 forwarding. One thing to note is that, you should use -x flag if you are not connecting to a trusted host ([read more here](#)). In my case all of my machines are behind a firewall and require VPN to access.

Get Reza Teshnizi's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

You should now be prompted to input your password to login. Once it is done, run `xeyes` which should result in this beauty:

```

reza@:~$ sudo su
Password:
Warning: No xauth data; using fake authentication data for X11 forwarding.
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 5.0.0-23-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:   https://landscape.canonical.com
 * Support:      https://ubuntu.com/advantage

 * Keen to Learn Istio? It's included in the single-package MicroK8s.
   https://snapcraft.io/microk8s

 * Canonical Livepatch is enabled.
   - All available patches applied.

10 packages can be updated.
0 updates are security updates.

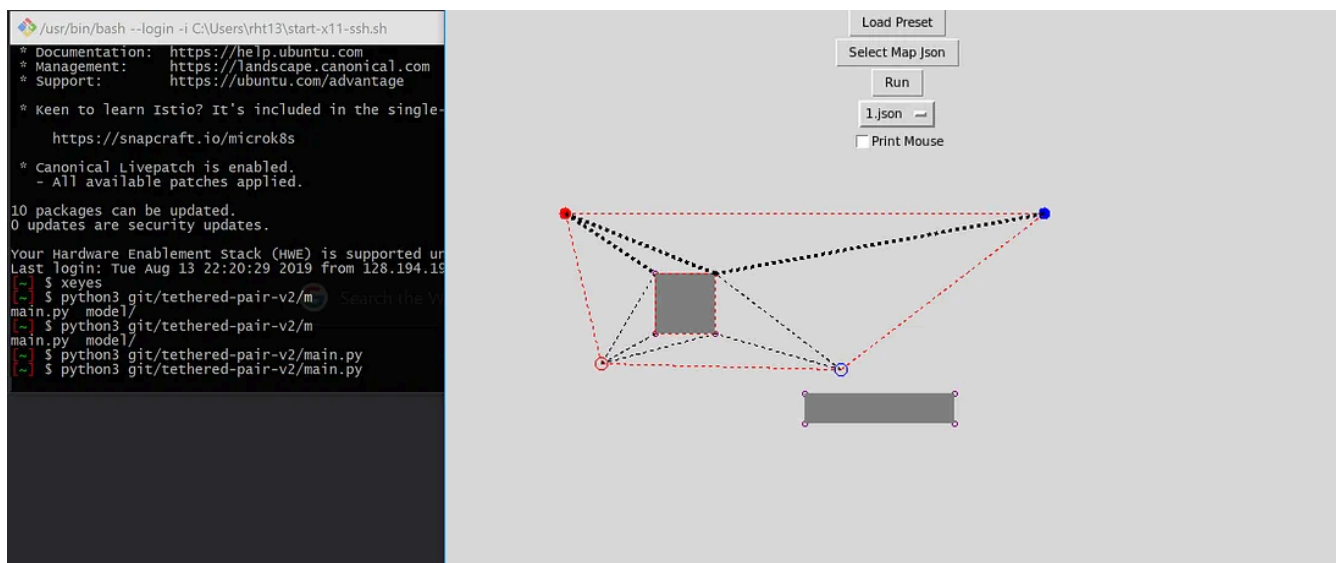
Your Hardware Enablement Stack (HWE) is supported until April 2023.
Last login: Tue Aug 13 22:20:29 2019 from 1
[~] $ xeyes

```

Look at those eyes

3) Run your Tkinter app remotely

At this point you have a way to remotely run your application and see the print statements for debugging.



My tkinter app running through X11 forwarding

If you don't have PyCharm or don't want to set it up, you are done.

PyCharm's Trick

Once again, you need a license for Professional/Student edition of PyCharm to be able to set this up (as of now). After installing and opening your project directory in PyCharm, you need to follow these steps to add your remote interpreter. In the interest of space I won't repeat those steps here.

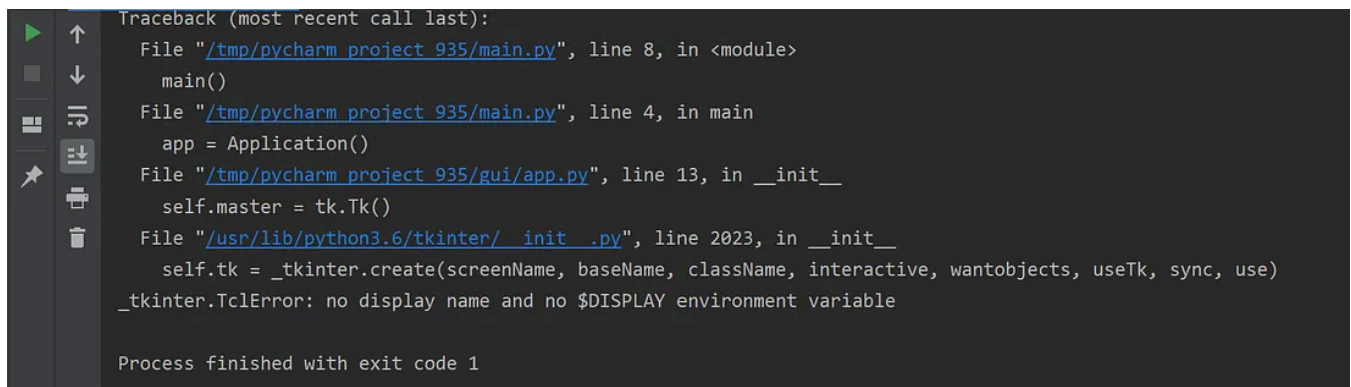
HOWEVER, there are two little tricks that enables you to run your GUI in PyCharm:

1) Before you open PyCharm you should open a Git Bash terminal, and run the two commands mentioned above. Once you are logged in, **leave that terminal open**.

```
export DISPLAY=localhost:0.0
ssh -Y <username>@<ip_address>
```

2) Now you need to add a DISPLAY environment variable to your PyCharm run configuration:

- Run your python file. This should give you an error in the console:

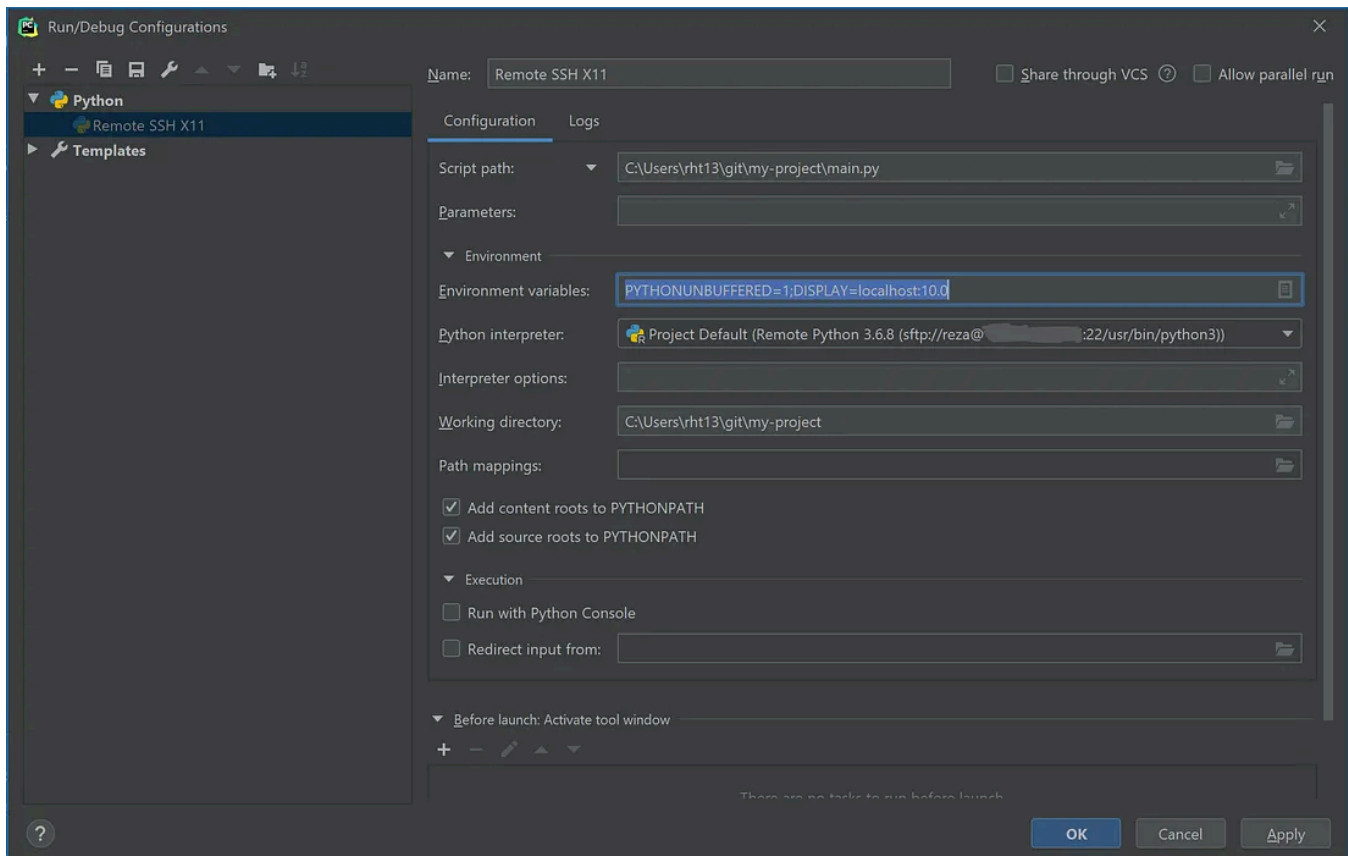
A screenshot of the PyCharm IDE's console window. The console shows a traceback error. The text in the console is as follows:

```
Traceback (most recent call last):
  File "/tmp/pycharm_project_935/main.py", line 8, in <module>
    main()
  File "/tmp/pycharm_project_935/main.py", line 4, in main
    app = Application()
  File "/tmp/pycharm_project_935/gui/app.py", line 13, in __init__
    self.master = tk.Tk()
  File "/usr/lib/python3.6/tkinter/_init_.py", line 2023, in __init__
    self.tk = _tkinter.create(screenName, baseName, className, interactive, wantobjects, useTk, sync, use)
_tkinter.TclError: no display name and no $DISPLAY environment variable

Process finished with exit code 1
```

no \$DISPLAY environment variable

- Now there should be a run configuration made for you. Go to Run > Edit Configurations...
- Select the configuration that you've created for this project.
- Add a DISPLAY environment variable with this value localhost:10.0 and press OK.



Environment Variables

Now hit **Alt+F5** and get back to work!!

Python

Tkinter

Ssh

X11

Pycharm



Follow

Written by **Reza Teshnizi**

10 followers · 1 following

Dog dad | PhD in Computer Science <https://rezateshnizi.com>

No responses yet





Write a response

What are your thoughts?

